OpenClaw Camera Application Tutorial

> Before starting, please complete the peripheral hardware configuration, API Key configuration, model configuration, and three dialogue solution configuration tests in the "Usage Preparation" directory. This article assumes the Jetson Nano Docker environment is already prepared and only covers CSI camera-related content.

1. Tutorial Objectives

This article demonstrates the photo and visual analysis capability of OpenClaw on Jetson Nano through the CSI camera peripheral. After completing this tutorial, readers will be able to:

- Understand the basic approach for OpenClaw on Jetson to use cameras
- Trigger photo capture and visual analysis via WhatsApp, TUI, and voice interaction
- Complete cases including photo archiving, scene description, OCR recognition, object observation, and more
- Understand the complete chain from "user submits a visual task" to "image capture" and "result return"

2. Hardware Preparation

- Jetson Nano B01
- OpenClaw Expansion Board
- CSI Camera Module
- Camera Cable
- `[Optional] AI Voice Interaction Module + Speaker`

Note:

- Camera cable direction must be correct; loose connectors will cause photo capture failure.
- Jetson CSI camera relies on system camera service; when abnormal, first execute `csi_snap snap` to check if camera is normal.
- Visual analysis usually requires properly configured model and API Key.

3. Software Preparation

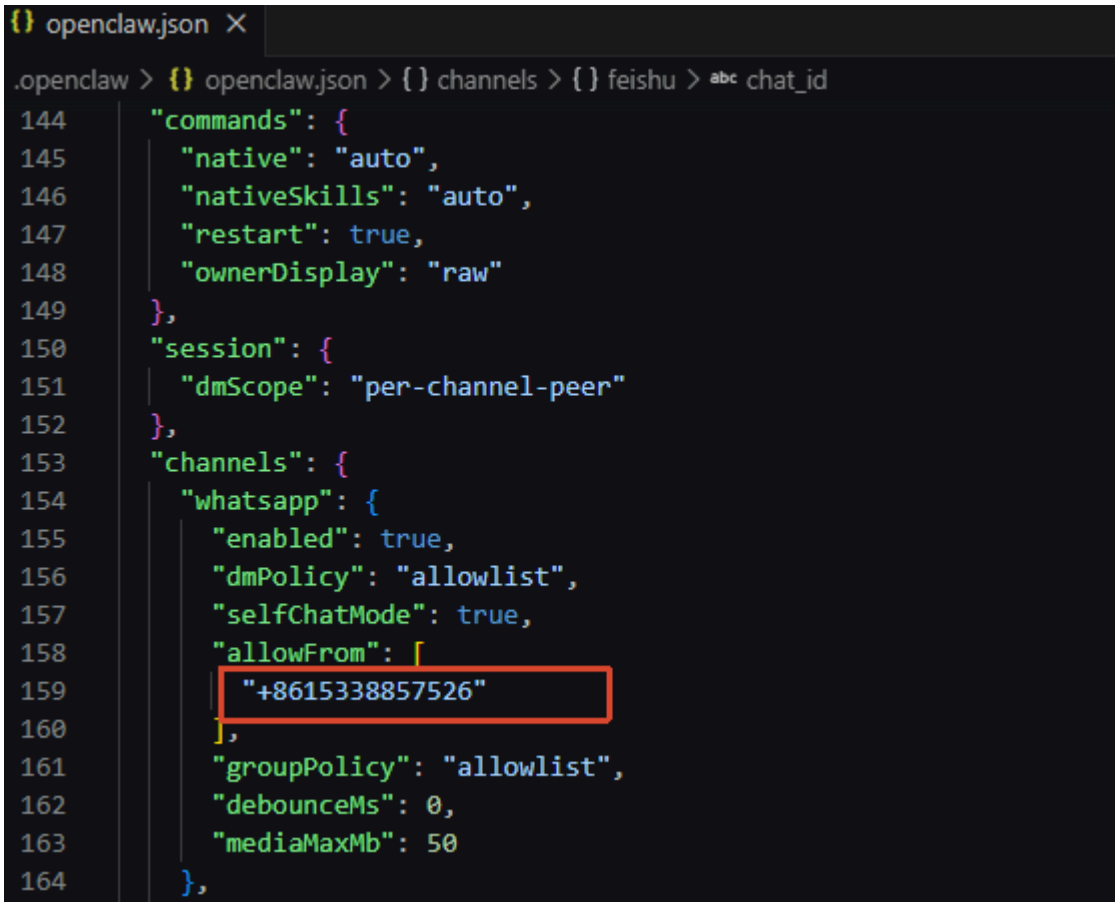
Enter the Jetson Docker container:

```
~/run_usb_docker.sh
```

Open the newly created conversation to see the conversation ID

Fill this into the "allowFrom" under the openclaw.json file:

```
/root/.openclaw/openclaw.json
```



Execute the following command in terminal to restart openclaw gateway:

```
openclaw gateway restart
```

You can use the following command to view current openclaw related processes:

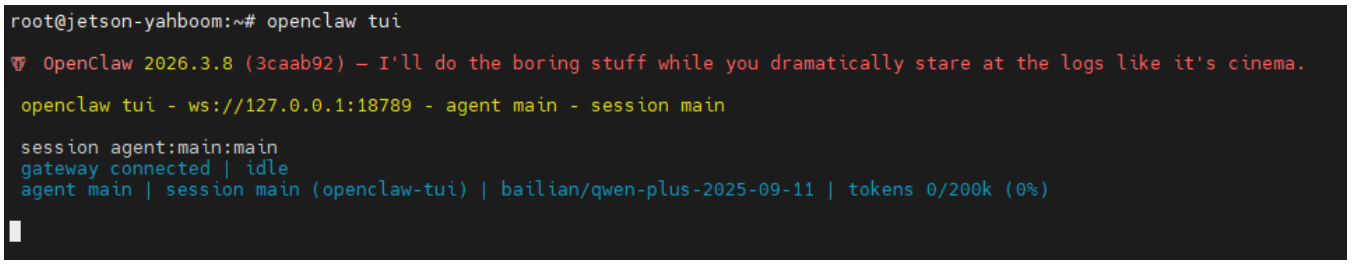
```
ps aux | grep openclaw
```

4. Testing Camera Application

Before starting this tutorial, please complete a basic `openclaw tui` dialogue self-check. Also please confirm that the visual model and API Key are already configured.

Enter TUI by typing the following command in the terminal:

```
openclaw tui
```



In the terminal, you can try inputting the following:

- Take a photo for me
- Describe the scene in front of the lens
- Recognize the text in the image

5. Three Methods for Dialogue with OpenClaw

Method	Suitable Scenario	Advantages	Recommended Use Case
WhatsApp	Remote viewing of images, classroom demonstration	Convenient on mobile, suitable for multi-person experience	Demonstrate remote visual capability
TUI	Local debugging, command verification	Most direct, easiest to troubleshoot	First establish the visual link
Voice Interaction	Voice observation, complete AI experience	Closest to real dialogue experience	Final demonstration solution

5.1 WhatsApp Solution

For WhatsApp access, QR code creation, and gateway startup steps, please refer to "Three Dialogue Solution Configuration Tests". This section only includes dialogue examples suitable for the camera tutorial.

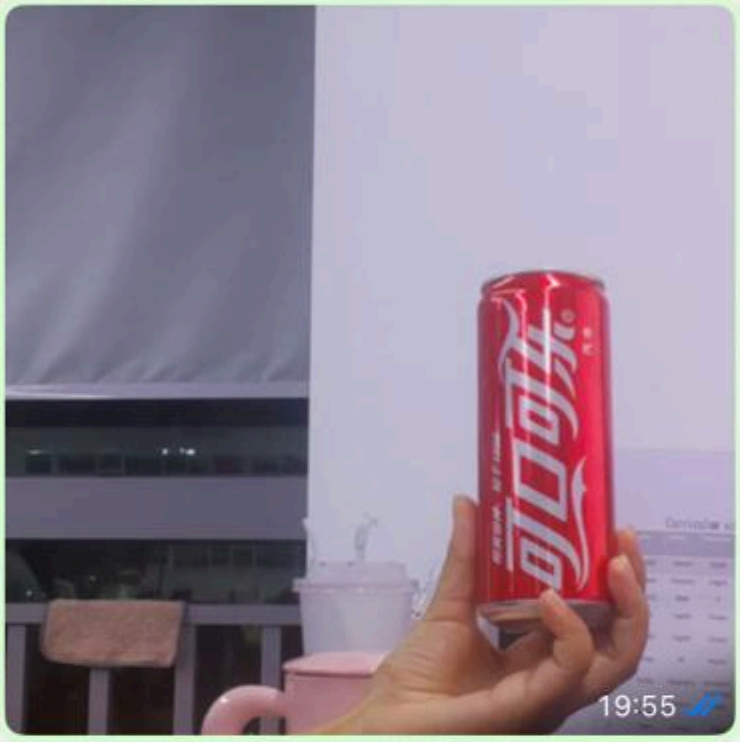
You can directly have conversations, for example:

- Look at what's in front
- After taking a photo, tell me what's on the desk
- Take a photo, fully recognize the text in the image, and organize it into a table and send it to me

19:55

< 2 +86 153 3885 7526 (你)
给自己发消息

Look at what's in front of 19:55



I can see the scene in front of the camera:

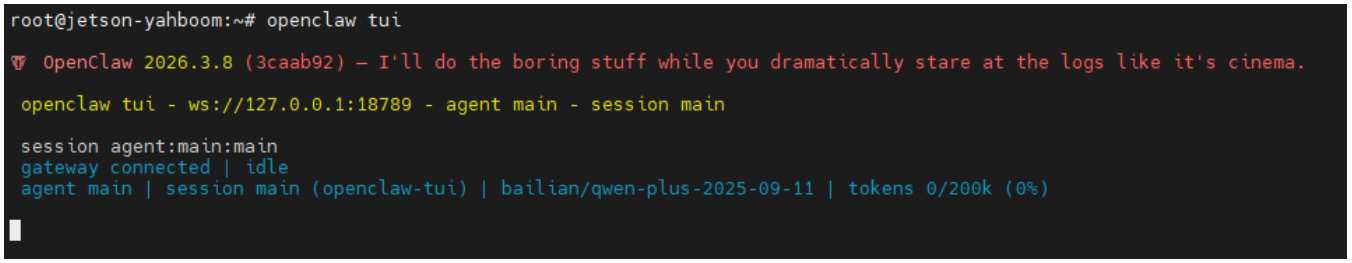
- A hand is holding a **red Coca-Cola can** in the center
- To the right: a small **green desk fan** and a **water bottle**
- In the background: a **curriculum schedule** paper posted on the wall
- On the left: what looks like a **purple phone case** and a **window with blinds**
- Various items on the desk including a **pink mug** and some containers

The scene appears to be a desk or

5.2 TUI Solution

If TUI self-check is already complete, directly type in the container:

```
openclaw tui
```



Then input camera-related commands:

- Take a photo of the current screen
- Describe the scene in front of the lens
- Recognize text in the screen
- Take a photo and analyze what's in front

5.3 Voice Interaction Solution

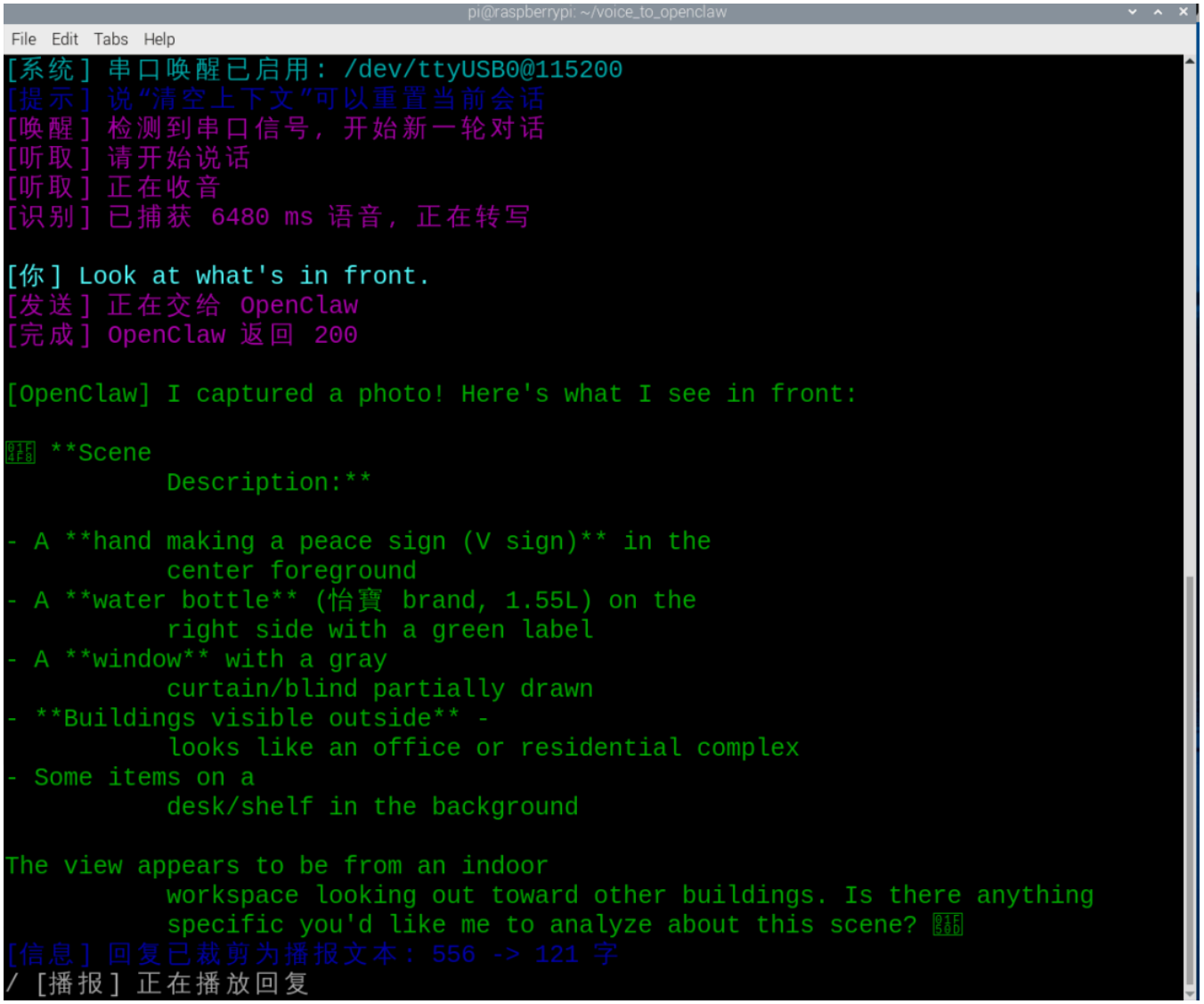
Requires optional AI Voice Interaction Module and speaker.

Execute the following command with openclaw gateway running:

```
cd /root/yahboom_ws/src/largemodel
python3 main.py
```

Suitable for directly demonstrating camera voice commands, for example:

- Take a photo for me
- Look at what's in front
- Let's play rock-paper-scissors



If the voice has recognized the text but no photo was taken, first check whether `openclaw_bridge` has forwarded the voice text to OpenClaw, and whether OpenClaw has generated a camera action.

6. Comprehensive Cases

The following cases can be executed in TUI, WhatsApp, or voice. It is recommended to use TUI first during the debugging phase.

6.1 Case 1: Photo Archiving

Experiment Objective

Have OpenClaw complete the most basic photo function and save the image.

Example Commands

- Take a photo for me
- Capture the current screen
- Take a photo and save it now

6.2 Case 2: Scene Description

Experiment Objective

Have OpenClaw provide a brief description of the current screen after taking a photo.

Example Commands

- See what's in front
- Help me describe the current screen
- After taking a photo, tell me what's mainly in the lens

6.3 Case 3: OCR Recognition

Experiment Objective

Have OpenClaw recognize text in images and return through natural language.

Example Commands

- Recognize the text in the image
- Take a photo and read the content on the paper
- Take a photo and organize the text inside

6.4 Case 4: Object Observation

Experiment Objective

Have OpenClaw observe specific objects in the screen, such as cups, gestures, pet food, etc.

Example Commands

- Find the cup in the screen
- See how much dog food is left
- Take a photo to recognize my gesture and see what I played

6.5 Case 5: Visual Linkage

Experiment Objective

After photo analysis, link with OLED, RGB, or servo to form a complete interaction process.

Example Commands

- Food calorie detection
- Let's play rock-paper-scissors
- See how much dog food is left

7. Common Problems

- Photo capture failed: First execute `csi_snap snap` to confirm if the camera is functioning normally.